

documentation

1.0

Generated by Doxygen 1.17.0

Глава 1

Group1

Глава 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Database< T >	??
Database< Employee >	??
EmployeeDB	??
Database< Task >	??
TaskDB	??
Employee	??
Task	??
WorkTimeSystem	??

Глава 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Database< T >	??
Employee	
Структура, представляющая сотрудника	??
EmployeeDB	
Класс базы данных для сотрудников (Employee (p. ??))	??
Task	
Структура, представляющая задачу	??
TaskDB	
Класс базы данных для задач (Task (p. ??))	??
WorkTimeSystem	??

Глава 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

database.h	...	??
employee.cpp		
Реализация класса Employee (p. ??)	...	??
employee.h		
Класс сотрудника для системы управления задачами	...	??
main.cpp		
Точка входа в программу "Система учёта рабочего времени"	...	??
system.cpp		
Реализация класса WorkTimeSystem (p. ??) - системы учёта рабочего времени	...	??
system.h		
Класс WorkTimeSystem (p. ??) - основная логика приложения	...	??
task.cpp		
Реализация класса Task (p. ??)	...	??
task.h		
Класс задачи (Task (p. ??))	...	??

Глава 5

Class Documentation

5.1 Database< T > Class Template Reference

Public Member Functions

- Database (const std::string &file)
Конструктор базы данных
- virtual ~Database ()=default
Виртуальный деструктор для корректного освобождения ресурсов в наследниках
- void load ()
Загружает данные из файла в память
- void save () const
Сохраняет текущее состояние базы данных в файл
- void add (const T &item)
Добавляет новый объект в базу данных
- bool remove (int id)
Удаляет объект по идентификатору
- T * find (int id)
Находит объект по идентификатору
- const T * find (int id) const
Константный поиск объекта по идентификатору
- std::vector< T > & getAll ()
Возвращает ссылку на вектор всех объектов
- const std::vector< T > & getAll () const
Константная версия getAll() (p. ??).
- size_t size () const
Возвращает количество объектов в базе
- bool empty () const
Проверяет, пуста ли база данных
- void clear ()
Очищает все данные из памяти (без удаления файла).
- void reload ()
Перезагружает базу данных из файла (очищает память + загружает).

Protected Member Functions

- `virtual std::string serialize (const T &item) const =0`
Сериализует объект в строку для сохранения в файл
- `virtual T deserialize (const std::string &data) const =0`
Десериализует объект из строки, прочитанной из файла

5.1.1 Constructor & Destructor Documentation

5.1.1.1 Database()

```
template<typename T>
Database< T >::Database (
    const std::string & file) [inline]
```

Конструктор базы данных

Parameters

file	Путь к файлу для хранения данных
------	----------------------------------

5.1.2 Member Function Documentation

5.1.2.1 add()

```
template<typename T>
void Database< T >::add (
    const T & item) [inline]
```

Добавляет новый объект в базу данных

Parameters

item	Добавляемый объект
------	--------------------

Note

Автоматически сохраняет изменения в файл

5.1.2.2 clear()

```
template<typename T>
void Database< T >::clear () [inline]
```

Очищает все данные из памяти (без удаления файла).

Warning

Чтобы сохранить очистку, после вызова нужно вызвать `save()` (p. ??)

5.1.2.3 deserialize()

```
template<typename T>
virtual T Database< T >::deserialize (
    const std::string & data) const    [protected], [pure virtual]
```

Десериализует объект из строки, прочитанной из файла

Parameters

data	Строка из файла
------	-----------------

Returns

Восстановленный объект

Note

Метод должен быть переопределён в классе-наследнике

Implemented in EmployeeDB (p.??), and TaskDB (p.??).

5.1.2.4 empty()

```
template<typename T>
bool Database< T >::empty () const    [inline]
```

Проверяет, пуста ли база данных

Returns

true если база не содержит ни одного объекта

5.1.2.5 find() [1/2]

```
template<typename T>
T * Database< T >::find (
    int id)    [inline]
```

Находит объект по идентификатору

Parameters

id	Идентификатор объекта
----	-----------------------

Returns

Указатель на найденный объект или nullptr, если не найден

Warning

Возвращает указатель на внутренние данные — изменение объекта через этот указатель изменяет данные в базе, но для сохранения в файл потребуется вызвать save() (p.??) вручную.

5.1.2.6 find() [2/2]

```
template<typename T>
const T * Database< T >::find (
    int id) const [inline]
```

Константный поиск объекта по идентификатору

Parameters

id	Идентификатор объекта
----	-----------------------

Returns

Константный указатель на найденный объект или nullptr

5.1.2.7 getAll() [1/2]

```
template<typename T>
std::vector< T > & Database< T >::getAll () [inline]
```

Возвращает ссылку на вектор всех объектов

Returns

Ссылка на вектор объектов

5.1.2.8 getAll() [2/2]

```
template<typename T>
const std::vector< T > & Database< T >::getAll () const [inline]
```

Константная версия getAll() (p. ??).

Returns

Константная ссылка на вектор объектов

5.1.2.9 load()

```
template<typename T>
void Database< T >::load () [inline]
```

Загружает данные из файла в память

Читает файл построчно и десериализует каждую непустую строку в объект. Если файл не существует, метод просто возвращает управление.

Warning

Предыдущие данные в векторе НЕ очищаются — повторный вызов load() (p. ??) приведёт к дублированию записей.

Note

Для полной перезагрузки перед вызовом вызовите clear() (p. ??)

5.1.2.10 remove()

```
template<typename T>
bool Database< T >::remove (
    int id) [inline]
```

Удаляет объект по идентификатору

Parameters

id	Идентификатор удаляемого объекта
----	----------------------------------

Returns

true если объект найден и удалён, false если объект с таким id не найден

Note

Автоматически сохраняет изменения в файл

5.1.2.11 save()

```
template<typename T>
void Database< T >::save () const [inline]
```

Сохраняет текущее состояние базы данных в файл

Перезаписывает файл, записывая каждый объект в отдельную строку через метод `serialize()` (p. ??).

5.1.2.12 serialize()

```
template<typename T>
virtual std::string Database< T >::serialize (
    const T & item) const [protected], [pure virtual]
```

Сериализует объект в строку для сохранения в файл

Parameters

item	Объект для сериализации
------	-------------------------

Returns

Строковое представление объекта

Note

Метод должен быть переопределён в классе-наследнике

Implemented in `EmployeeDB` (p. ??), and `TaskDB` (p. ??).

5.1.2.13 size()

```
template<typename T>  
size_t Database< T >::size () const [inline]
```

Возвращает количество объектов в базе

Returns

Размер базы данных

The documentation for this class was generated from the following file:

- database.h

5.2 Employee Struct Reference

Структура, представляющая сотрудника

```
#include <employee.h>
```

Public Member Functions

- Employee ()
Конструктор по умолчанию
- Employee (int id, const std::string &name, float hoursPerWeek)
Конструктор с параметрами
- std::string serialize () const

Static Public Member Functions

- static Employee deserialize (const std::string &data)
Восстанавливает сотрудника из строки

Public Attributes

- int id
Уникальный идентификатор сотрудника
- std::string fullName
Полное имя сотрудника
- float hoursPerWeek
Количество рабочих часов в неделю
- std::vector< int > assignedTaskIds
Список ID задач, назначенных сотруднику

5.2.1 Detailed Description

Структура, представляющая сотрудника

Хранит информацию о сотруднике и список ID назначенных ему задач. Поддерживает сериализацию в строку и обратное восстановление.

Note

Поле id должно быть уникальным для каждого сотрудника

See also

Task (p. ??)

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Employee() [1/2]

Employee::Employee ()

Конструктор по умолчанию

Postcondition

id = 0, hoursPerWeek = 0, fullName и assignedTaskIds пусты

5.2.2.2 Employee() [2/2]

```
Employee::Employee (  
    int id,  
    const std::string & name,  
    float hoursPerWeek)
```

Конструктор с параметрами

Parameters

id	Уникальный идентификатор
name	Полное имя
hoursPerWeek	Часы работы в неделю

5.2.3 Member Function Documentation

5.2.3.1 deserialize()

```
Employee Employee::deserialize (  
    const std::string & data) [static]
```

Восстанавливает сотрудника из строки

Parameters

data	Строка в формате serialize()
------	------------------------------

Returns

Восстановленный объект Employee (p. ??)

See also

serialize()

The documentation for this struct was generated from the following files:

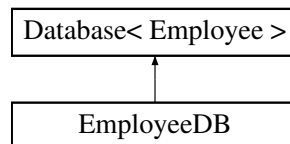
- employee.h
- employee.cpp

5.3 EmployeeDB Class Reference

Класс базы данных для сотрудников (Employee (p. ??)).

```
#include <database.h>
```

Inheritance diagram for EmployeeDB:



Public Member Functions

- EmployeeDB (const std::string &file)
Конструктор базы данных сотрудников

Public Member Functions inherited from Database< Employee >

- Database (const std::string &file)
Конструктор базы данных
- virtual ~Database ()=default
Виртуальный деструктор для корректного освобождения ресурсов в наследниках
- void load ()
Загружает данные из файла в память
- void save () const
Сохраняет текущее состояние базы данных в файл
- void add (const Employee &item)
Добавляет новый объект в базу данных
- bool remove (int id)
Удаляет объект по идентификатору
- Employee * find (int id)
Находит объект по идентификатору
- std::vector< Employee > & getAll ()
Возвращает ссылку на вектор всех объектов
- size_t size () const
Возвращает количество объектов в базе
- bool empty () const
Проверяет, пуста ли база данных
- void clear ()
Очищает все данные из памяти (без удаления файла).
- void reload ()
Перезагружает базу данных из файла (очищает память + загружает).

Protected Member Functions

- std::string serialize (const Employee &item) const override
Сериализует сотрудника в строку
- Employee deserialize (const std::string &data) const override
Десериализует сотрудника из строки

5.3.1 Detailed Description

Класс базы данных для сотрудников (Employee (p.??)).

Специализация шаблонного класса Database (p.??) для работы с объектами Employee (p.??). Реализует методы сериализации через соответствующие методы класса Employee (p.??).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 EmployeeDB()

```
EmployeeDB::EmployeeDB (
    const std::string & file) [inline]
```

Конструктор базы данных сотрудников

Parameters

file	Путь к файлу (например, "employees.txt")
------	--

5.3.3 Member Function Documentation

5.3.3.1 deserialize()

```
Employee EmployeeDB::deserialize (
    const std::string & data) const [inline], [override], [protected], [virtual]
```

Десериализует сотрудника из строки

Parameters

data	Строка из файла
------	-----------------

Returns

Объект Employee (p. ??), восстановленный через Employee::deserialize() (p. ??)

Implements Database< Employee > (p. ??).

5.3.3.2 serialize()

```
std::string EmployeeDB::serialize (
    const Employee & item) const [inline], [override], [protected], [virtual]
```

Сериализует сотрудника в строку

Parameters

item	Объект Employee (p. ??)
------	-------------------------

Returns

Строка, полученная из Employee::serialize()

Implements Database< Employee > (p. ??).

The documentation for this class was generated from the following file:

- database.h

5.4 Task Struct Reference

Структура, представляющая задачу

```
#include <task.h>
```

Public Member Functions

- Task ()
Конструктор по умолчанию (id=0, все поля нулевые, status="не начата").
- Task (int id, const std::string &desc, float hoursReq, int deadlineH)
Конструктор с параметрами
- std::string serialize () const

Static Public Member Functions

- static Task deserialize (const std::string &data)
Восстанавливает задачу из строки (формат serialize).

Public Attributes

- int id
Уникальный идентификатор задачи
- std::string description
Описание задачи
- float hoursRequired
Требуемое количество часов
- float hoursSpent
Уже затраченные часы
- int deadlineHours
Срок выполнения (часов от создания).
- std::string status
Статус: "не начата", "в процессе", "готова", "просрочена".

5.4.1 Detailed Description

Структура, представляющая задачу

Хранит информацию о задаче: описание, трудоёмкость, затраченное время, дедлайн и текущий статус выполнения. Поддерживает сериализацию.

5.4.2 Member Function Documentation

5.4.2.1 serialize()

```
std::string Task::serialize () const
```

Returns

Строка в формате "id|description|hoursRequired|hoursSpent|deadlineHours|status"

The documentation for this struct was generated from the following files:

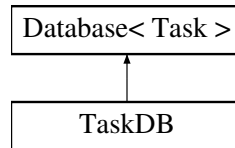
- task.h
- task.cpp

5.5 TaskDB Class Reference

Класс базы данных для задач (Task (p. ??)).

```
#include <database.h>
```

Inheritance diagram for TaskDB:



Public Member Functions

- TaskDB (const std::string &file)
Конструктор базы данных задач

Public Member Functions inherited from Database< Task >

- Database (const std::string &file)
Конструктор базы данных
- virtual ~Database ()=default
Виртуальный деструктор для корректного освобождения ресурсов в наследниках
- void load ()
Загружает данные из файла в память
- void save () const
Сохраняет текущее состояние базы данных в файл
- void add (const Task &item)
Добавляет новый объект в базу данных
- bool remove (int id)
Удаляет объект по идентификатору
- Task * find (int id)
Находит объект по идентификатору
- std::vector< Task > & getAll ()
Возвращает ссылку на вектор всех объектов
- size_t size () const
Возвращает количество объектов в базе
- bool empty () const
Проверяет, пуста ли база данных
- void clear ()
Очищает все данные из памяти (без удаления файла).
- void reload ()
Перезагружает базу данных из файла (очищает память + загружает).

Protected Member Functions

- std::string serialize (const Task &item) const override
Сериализует задачу в строку
- Task deserialize (const std::string &data) const override
Десериализует задачу из строки

5.5.1 Detailed Description

Класс базы данных для задач (Task (p. ??)).

Специализация шаблонного класса Database (p. ??) для работы с объектами Task (p. ??). Реализует методы сериализации через соответствующие методы класса Task (p. ??).

5.5.2 Constructor & Destructor Documentation

5.5.2.1 TaskDB()

```
TaskDB::TaskDB (  
    const std::string & file) [inline]
```

Конструктор базы данных задач

Parameters

file	Путь к файлу (например, "tasks.txt")
------	--------------------------------------

5.5.3 Member Function Documentation

5.5.3.1 deserialize()

```
Task TaskDB::deserialize (  
    const std::string & data) const [inline], [override], [protected], [virtual]
```

Десериализует задачу из строки

Parameters

data	Строка из файла
------	-----------------

Returns

Объект Task (p. ??), восстановленный через Task::deserialize() (p. ??)

Implements Database< Task > (p. ??).

5.5.3.2 serialize()

```
std::string TaskDB::serialize (
    const Task & item) const    [inline], [override], [protected], [virtual]
```

Сериализует задачу в строку

Parameters

item	Объект Task (p. ??)
------	---------------------

Returns

Строка, полученная из Task::serialize() (p. ??)

Implements Database< Task > (p. ??).

The documentation for this class was generated from the following file:

- database.h

5.6 WorkTimeSystem Class Reference

Public Member Functions

- void addEmployee (const std::string &name, float hoursPerWeek)
- void addTask (const std::string &desc, float hoursRequired, int deadlineHours)
- void removeEmployee (int id)
- void removeTask (int id)
- void assignTask (int taskId, int employeeId)
- void unassignTask (int taskId, int employeeId)
- void viewAllEmployees () const
- void viewAllTasks () const
- void viewEmployeeTasks (int employeeId) const
- void advanceTime (int days)
- void updateTaskStatus ()
- void runMenu ()

The documentation for this class was generated from the following files:

- system.h
- system.cpp

Глава 6

File Documentation

6.1 database.h

```
00001
00007
00008 #ifndef DATABASE_HPP
00009 #define DATABASE_HPP
00010
00011 #include <vector>
00012 #include <string>
00013 #include <fstream>
00014 #include <algorithm>
00015
00048 template<typename T>
00049 class Database {
00050 private:
00051     std::vector<T> items;
00052     std::string filename;
00053
00054 protected:
00062     virtual std::string serialize(const T& item) const = 0;
00063
00071     virtual T deserialize(const std::string& data) const = 0;
00072
00073 public:
00078     Database(const std::string& file) : filename(file) {}
00079
00083     virtual ~Database() = default;
00084
00096     void load() {
00097         std::ifstream in(filename);
00098         if (!in.is_open()) return;
00099
00100         std::string line;
00101         while (std::getline(in, line)) {
00102             if (!line.empty()) {
00103                 items.push_back(deserialize(line));
00104             }
00105         }
00106         in.close();
00107     }
00108
00115     void save() const {
00116         std::ofstream out(filename);
00117         for (const auto& item : items) {
00118             out << serialize(item) << std::endl;
00119         }
00120         out.close();
00121     }
00122
00129     void add(const T& item) {
00130         items.push_back(item);
00131         save();
00132     }
00133
00141     bool remove(int id) {
00142         auto it = std::find_if(items.begin(), items.end(),
00143             [id](const T& item) { return item.id == id; });
00144         if (it != items.end()) {
00145             items.erase(it);
```

```

00146         save();
00147         return true;
00148     }
00149     return false;
00150 }
00151
00161 T* find(int id) {
00162     auto it = std::find_if(items.begin(), items.end(),
00163         [id](const T& item) { return item.id == id; });
00164     if (it != items.end()) {
00165         return &(*it);
00166     }
00167     return nullptr;
00168 }
00169
00175 const T* find(int id) const {
00176     auto it = std::find_if(items.begin(), items.end(),
00177         [id](const T& item) { return item.id == id; });
00178     if (it != items.end()) {
00179         return &(*it);
00180     }
00181     return nullptr;
00182 }
00183
00188 std::vector<T>& getAll() { return items; }
00189
00194 const std::vector<T>& getAll() const { return items; }
00195
00200 size_t size() const { return items.size(); }
00201
00206 bool empty() const { return items.empty(); }
00207
00213 void clear() { items.clear(); }
00214
00218 void reload() {
00219     items.clear();
00220     load();
00221 }
00222 };
00223
00230 class TaskDB : public Database<Task> {
00231 protected:
00237     std::string serialize(const Task& item) const override {
00238         return item.serialize();
00239     }
00240
00246     Task deserialize(const std::string& data) const override {
00247         return Task::deserialize(data);
00248     }
00249
00250 public:
00255     TaskDB(const std::string& file) : Database<Task>(file) {}
00256 };
00257
00264 class EmployeeDB : public Database<Employee> {
00265 protected:
00271     std::string serialize(const Employee& item) const override {
00272         return item.serialize();
00273     }
00274
00280     Employee deserialize(const std::string& data) const override {
00281         return Employee::deserialize(data);
00282     }
00283
00284 public:
00289     EmployeeDB(const std::string& file) : Database<Employee>(file) {}
00290 };
00291
00292 #endif // DATABASE_HPP

```

6.2 employee.cpp File Reference

Реализация класса Employee (p. ??).

```

#include "employee.h"
#include <sstream>

```

6.2.1 Detailed Description

Реализация класса Employee (p. ??).

6.3 employee.h File Reference

Класс сотрудника для системы управления задачами

```
#include <string>
#include <vector>
```

Classes

- struct Employee
Структура, представляющая сотрудника

6.3.1 Detailed Description

Класс сотрудника для системы управления задачами

6.4 employee.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef EMPLOYEE_H
00007 #define EMPLOYEE_H
00008
00009 #include <string>
00010 #include <vector>
00011
00021 struct Employee {
00022     int id;
00023     std::string fullName;
00024     float hoursPerWeek;
00025     std::vector<int> assignedTaskIds;
00026
00031     Employee();
00032
00039     Employee(int id, const std::string& name, float hoursPerWeek);
00040
00050     std::string serialize() const;
00051
00058     static Employee deserialize(const std::string& data);
00059 };
00060
00061 #endif // EMPLOYEE_H
```

6.5 main.cpp File Reference

Точка входа в программу "Система учёта рабочего времени".

```
#include "system.h"
#include <locale>
```

Functions

- `int main ()`
Главная функция программы

6.5.1 Detailed Description

Точка входа в программу "Система учёта рабочего времени".

6.5.2 Function Documentation

6.5.2.1 `main()`

`int main ()`

Главная функция программы

Устанавливает русскую локаль для корректного отображения кириллицы в консоли, создаёт экземпляр системы и запускает главное меню.

Returns

0 при успешном завершении

6.6 `system.cpp` File Reference

Реализация класса `WorkTimeSystem` (р. ??) - системы учёта рабочего времени

```
#include "system.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
```

6.6.1 Detailed Description

Реализация класса `WorkTimeSystem` (р. ??) - системы учёта рабочего времени

6.7 `system.h` File Reference

Класс `WorkTimeSystem` (р. ??) - основная логика приложения

```
#include "task.h"
#include "employee.h"
#include "database.h"
#include <memory>
```

Classes

- class WorkTimeSystem

6.7.1 Detailed Description

Класс WorkTimeSystem (p. ??) - основная логика приложения

6.8 system.h

Go to the documentation of this file.

```

00001
00005
00006 #ifndef SYSTEM_H
00007 #define SYSTEM_H
00008
00009 #include "task.h"
00010 #include "employee.h"
00011 #include "database.h"
00012 #include <memory>
00013
00014 class WorkTimeSystem {
00015 private:
00016     std::unique_ptr<TaskDB> taskDB;
00017     std::unique_ptr<EmployeeDB> employeeDB;
00018     int nextTaskId;
00019     int nextEmployeeId;
00020
00021 public:
00022     WorkTimeSystem();
00023
00024     void addEmployee(const std::string& name, float hoursPerWeek);
00025     void addTask(const std::string& desc, float hoursRequired, int deadlineHours);
00026     void removeEmployee(int id);
00027     void removeTask(int id);
00028     void assignTask(int taskId, int employeeId);
00029     void unassignTask(int taskId, int employeeId);
00030     void viewAllEmployees() const;
00031     void viewAllTasks() const;
00032     void viewEmployeeTasks(int employeeId) const;
00033     void advanceTime(int days);
00034     void updateTaskStatus();
00035     void runMenu();
00036
00037 private:
00038     void loadData();
00039     void saveData();
00040 };
00041
00042 #endif

```

6.9 task.cpp File Reference

Реализация класса Task (p. ??).

```

#include "task.h"
#include <sstream>

```

6.9.1 Detailed Description

Реализация класса Task (p. ??).

6.10 task.h File Reference

Класс задачи (Task (p.??)).

```
#include <string>
#include <vector>
```

Classes

- struct Task
Структура, представляющая задачу

6.10.1 Detailed Description

Класс задачи (Task (p.??)).

6.11 task.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef TASK_H
00007 #define TASK_H
00008
00009 #include <string>
00010 #include <vector>
00011
00018 struct Task {
00019     int id;
00020     std::string description;
00021     float hoursRequired;
00022     float hoursSpent;
00023     int deadlineHours;
00024     std::string status;
00025
00027     Task();
00028
00030     Task(int id, const std::string& desc, float hoursReq, int deadlineH);
00031
00033     std::string serialize() const;
00034
00036     static Task deserialize(const std::string& data);
00037 };
00038
00039 #endif // TASK_H
```

Глава 7

Examples

7.1 C:/Users/levvis/OneDrive/Рабочий стол/fastapi_practice/anya/Group1/database.h

Шаблонный абстрактный класс для работы с базой данных

Шаблонный абстрактный класс для работы с базой данных Предоставляет базовый функционал CRUD (Create, Read, Update, Delete) для хранения объектов в текстовом файле. Каждая запись хранится в отдельной строке.

Template Parameters

T	Тип объектов, хранящихся в базе. Должен иметь поле id и методы serialize()/deserialize().
---	---

Note

Класс является абстрактным: необходимо переопределить методы serialize() и deserialize()

```
class MyData {
public:
    int id;
    std::string name;
    std::string serialize() const { return std::to_string(id) + "|" + name; }
    static MyData deserialize(const std::string& data) { /* ... */ }
};

class MyDB : public Database<MyData> {
protected:
    std::string serialize(const MyData& item) const override { return item.serialize(); }
    MyData deserialize(const std::string& data) const override { return MyData::deserialize(data); }
};

MyDB db("data.txt");
db.load();
db.add(MyData{1, "example"});

```

стол/fastapi_practice/anya/Group1/database.h

7.2 C:/Users/levvis/OneDrive/Рабочий стол/fastapi_practice/anya/Group1/employee.h

Сериализует сотрудника в строку

Сериализует сотрудника в строку

Returns

Формат: "id|fullName|hoursPerWeek|taskId1,taskId2,..."

```
Employee (p.??) e(5, "Иван Петров", 40); e.assignedTaskIds = {1, 3, 7}; e.serialize(); // "5|Иван  
Петров|40|1,3,7"
```

стол/fastapi_practice/anya/Group1/employee.h